ANLC-752 · DATA MANAGEMENT · UNIT 8 MILESTONE

Structuring a Bitcoin transaction graph for anti-money-laundering monitoring

203,769 transactions · 234,355 money-flow edges · 49 periods · PostgreSQL 16

Alpesh Ranjan

Progress since Unit 1 · Elliptic Data Set

The question set in Unit 1

How can transaction-level Bitcoin data be structured in an RDBMS to support efficient identification and monitoring of patterns associated with illicit activity?

This is a data-management question, not a modelling one. The deliverable is a structure in which AML monitoring is expressible and auditable — so progress is measured by what the schema makes queryable, not by a model score.

Unit 1 — done

Research question fixed. Source evaluated for provenance, completeness and licence. Limitations stated up front rather than discovered later.

Unit 8 — this milestone

Data validated, loaded and normalised into PostgreSQL. Monitoring layer and analysis-preparation layer built entirely in SQL.

Unit 10 — final

Fit and evaluate a model on the prepared matrix. Extend to an actor layer. Implement the governance controls recommended here.

THE RAW DATA

What the dataset is, and why it fits

The Elliptic Data Set — real but anonymised Bitcoin transactions published by Elliptic Ltd. with the MIT-IBM Watson AI Lab. Every node is a transaction; every edge is bitcoin flowing from one transaction to another. Labels come from Elliptic's own compliance attribution process, not from the features, so feature-based analysis is not circular.

203,769
transactions (nodes)

234,355
directed money-flow edges

49
time periods, ~2 weeks each

22.85%
carry a ground-truth label

Genuinely relational

Nodes, directed edges, a wide attribute vector and an outcome map onto entities and relationships without contortion.

The graph is the point

Laundering is defined by movement. The edge table is what makes provenance tracing and exposure scoring expressible in SQL.

Real time axis

49 ordered periods support period-over-period risk profiling and a train/test split by time rather than at random.

Three files, one of them hostile

```
elliptic_txs_classes.csv     txId,class
                             230425980,unknown

elliptic_txs_edgelist.csv    txId1,txId2
                             230425980,5530458

elliptic_txs_features.csv    230425980,1,-0.1714692,...
                             (no header row)
```

File	Rows	Cols
classes	203,769	2
edgelist	234,355	2
features	203,769	167

The feature file ships with no header row and 167 unnamed columns. Nothing can be loaded until a schema is declared for it — which is why the schema is generated from a single constant rather than typed by hand.

Column layout of the feature file



Local features describe the transaction itself. Aggregate features are statistics over its immediate neighbours — the publisher pre-computed one hop of the graph, which is a hint that neighbourhood structure carries signal.

The data was already tidy — so cleaning meant something else

Profiled before assuming

Null values	0
Duplicate txIds	0
Orphan edge endpoints	0
Self-loops	0
Cross-period edges	0
Local features	already z-normalised

Verification, not imputation

Because nothing needed fixing, the burden shifts to proving that. Every property above is enforced as a named rule at load time and written to a governance table — so the claim "this data is clean" is queryable evidence, not an assertion in a slide.

There were no nulls to impute and no duplicates to drop. Claiming otherwise would have been fiction. The real obstacles to loading this data were structural, and each one is a decision on the record:

- **167 unnamed columns**

  schema declared from one constant, so core, staging and the COPY list cannot drift

- **class as mixed-type text**

  '1' / '2' / 'unknown' recoded to a typed dimension

- **77% unlabelled**

  kept in the graph, excluded from supervised scope — never coerced to licit

- **one 690 MB flat file**

  decomposed into a normalised relational model

VALIDATION

Nine rules, and proof they actually fire

Rule	Type	Sev
classes_row_count_nonzero	completeness	ERROR
classes_tx_id_unique	uniqueness	ERROR
classes_code_in_domain	domain	ERROR
features_tx_id_unique	uniqueness	ERROR
features_time_step_in_range	domain	ERROR
classes_features_tx_id_reconcile	reconciliation	ERROR
edgelist_endpoints_resolve	referential	ERROR
edgelist_no_self_loops	domain	WARN
features_no_null_values	completeness	ERROR

All nine passed on the full extract. The null check covers all 165 feature columns, not a sample.

A rule that never fails proves nothing

Three defects were injected into a copy of the input and the load re-run:

```
duplicate txId        caught
time_step = 99        caught
edge to missing tx    caught
```

Each was caught by its own named rule. The run aborted, core stayed empty, and the failure persisted to the governance table on a separate connection — so the abort is auditable rather than silent.

Four schemas, each with one job

staging

Untyped landing zone. Unlogged, truncated after every run. Nothing downstream reads it.

core

The conformed model. Typed, constrained, documented. transaction · tx_feature · tx_flow · 3 dimensions

analytics

Derived monitoring and analysis objects. 10 views and materialised views. Rebuildable from core.

governance

Load audit with SHA-256 of every source file, rule results, rejected rows.

Why separate them

A single schema would make "what did the source actually contain" and "what do we believe about it" the same question. Keeping staging, core and analytics apart means core can carry constraints that raw data would violate, and analytics can be dropped and rebuilt without touching evidence.

governance is deliberately outside that chain: it must survive a failed load, because a load that aborts is exactly when the audit trail matters.

Three decisions worth defending

Wide, not entity-attribute-value

165 features stored as 165 columns. EAV would turn 203,769 rows into 33.6M and force a pivot on every query — for no gain, since the feature set is fixed and non-sparse.

Unknown is not licit

dim_class carries is_labelled as a boolean separate from the class code. A class-3 transaction is one Elliptic could not attribute, not a negative example. The schema makes conflating them impossible.

Edges are transactions, not parties

tx_flow joins transaction to transaction. The dataset has no wallets or account holders, so no customer dimension is invented. The constraint is recorded in the table comment.

The schema is generated, not written

The 165 feature columns appear in the core table, the staging table and the bulk-load column list. Deriving all three from one constant in gen_schema.py removes the possibility of drift between them. The data dictionary is generated too — read back out of the live catalog after a load, so documentation cannot describe a database that was never built.

```
FEATURE_COLS = (
  [f"local_feature_{i}"     for i in 1..93]
+ [f"aggregate_feature_{i}" for i in 1..72]
)
# one source -> DDL, staging, COPY list
```

One transaction, or nothing

01	**Extract**	COPY into unlogged staging. SHA-256 of each source file recorded before a single row is read.
02	**Validate**	Nine rules evaluated against staging. Results written to governance regardless of outcome.
03	**Gate**	Any ERROR-severity failure raises. Nothing reaches core.
04	**Transform & load**	Recode class, populate dimensions, load facts. Edges deduplicated and self-loops dropped.
05	**Build**	Indexes, extended statistics, analytics and analysis layers.

Extract, validation and load run inside a single transaction. A failure at step 3 leaves the database exactly as it was — there is no partial state to reconcile and no half-loaded table to explain.

Measured

40–55 s

full load, extract through analytics. Spread is page-cache state on the 690 MB feature file, not the pipeline.

Idempotent

Re-running the load against a populated database rebuilds it rather than failing. A first run and a fifth run produce the same database — which is what makes the result reproducible by someone other than its author.

Filtering, and why the index matters

```
SELECT tx_id
FROM   core.transaction
WHERE  time_step = 32
  AND  class_id  = 1;    -- illicit
```

Result

342 illicit transactions in period 32

```
Index Scan using ix_transaction_labelled
  Index Cond: (class_id = 1 AND time_step = 32)
  Execution Time: 0.39 ms
```

A partial index, because 77% is noise

ix_transaction_labelled indexes only rows where the class is known. Three quarters of this table is unlabelled and no supervised query ever touches it, so indexing it would cost storage and write time to serve nothing.

The filter is on two columns whose selectivity is correlated — periods differ in how many illicit transactions they contain. Extended statistics on the pair tell the planner that, so it estimates the combination instead of multiplying two independent guesses.

0.39 ms on a 203,769-row table is the schema doing the work, not the hardware.

Turning edges into counterparty exposure

```
SELECT t.tx_id,
       COUNT(*) FILTER (WHERE n.class_id = 1) AS illicit_neighbours,
       COUNT(*) FILTER (WHERE n.class_id = 2) AS licit_neighbours
FROM   core.transaction t
JOIN   core.tx_flow     f ON f.src_tx_id = t.tx_id
                         OR f.dst_tx_id = t.tx_id
JOIN   core.transaction n ON n.tx_id = CASE
         WHEN f.src_tx_id = t.tx_id THEN f.dst_tx_id
         ELSE f.src_tx_id END
GROUP  BY t.tx_id;
```

The transaction table is joined to itself through the edge table, so a row about one transaction can carry facts about the transactions it touches. This is the join that makes counterparty risk expressible at all — without it, every transaction is an island and no laundering typology can be stated.

Materialised, not recomputed

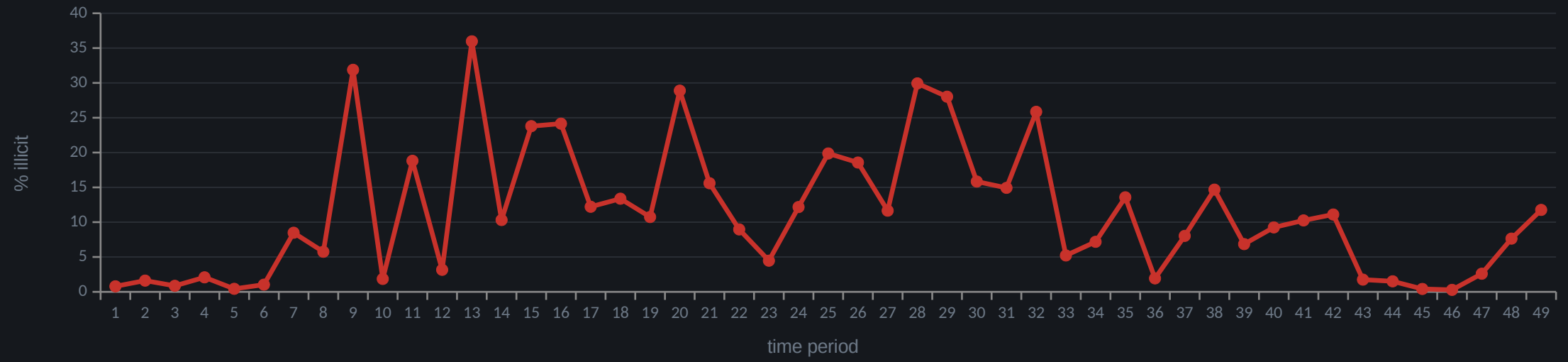
This join is the input to several monitoring views, so it is stored as mv_illicit_exposure rather than re-run each time. Refresh cost is bounded and stated.

Graph shape

472	highest fan-out
284	highest fan-in
2.30	mean degree
0	isolated nodes

One query, and the series changes shape

```
SELECT time_step, COUNT(*) FILTER (WHERE class_id=1)::NUMERIC / COUNT(*) ... GROUP BY time_step;
```

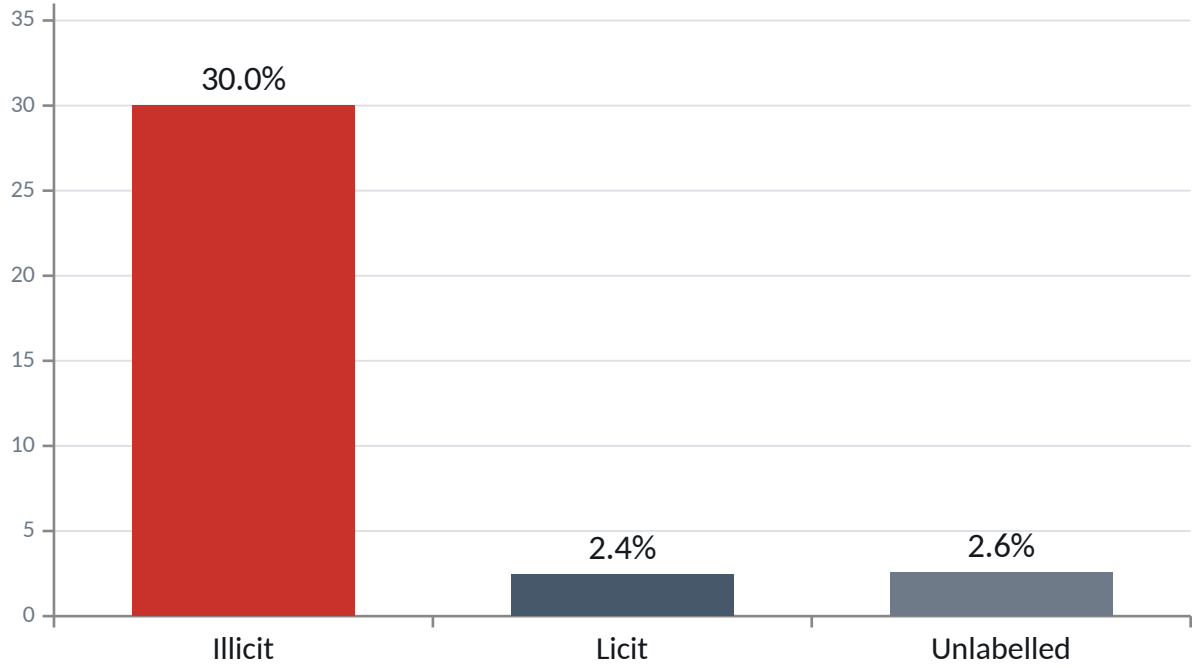


Periods 43–46

The illicit share of labelled transactions falls from 11.1% at period 42 to 0.28% at period 46 — a 40-fold drop across four consecutive periods, then a recovery to 11.8% by period 49.

This is a GROUP BY with an aggregate over one column. No model, no feature engineering, no statistical test — the structure of the loaded data is sufficient to surface a market-wide event that no individual transaction record contains. That is the case for building the database first.

Illicit transactions keep illicit company



Percentage of transactions with at least one known illicit counterparty, by their own class.

A 12× separation

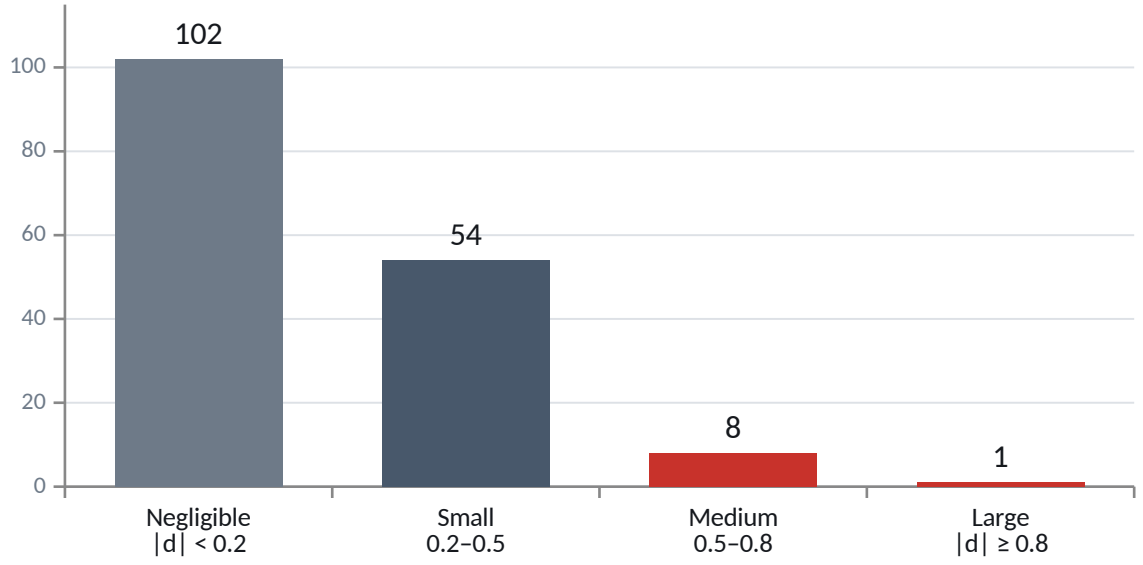
An illicit transaction is twelve times more likely than a licit one to sit next to another known illicit transaction. Money laundering is a structural phenomenon, and the structure is visible in a join.

The unlabelled population sits at the licit rate, not between the two — which is the empirical reason for refusing to treat "unknown" as a weak positive.

4,027 unlabelled transactions have at least one illicit counterparty. That set is vw_alert_queue: a triage list that exists because the edge table exists.

Which of the 165 features are worth giving a model

The features are obfuscated, so they cannot be selected on meaning. They can be selected on evidence: for each column, the difference in class means expressed in pooled standard deviations (Cohen's d), computed with GROUP BY, AVG and STDDEV over the labelled subset.

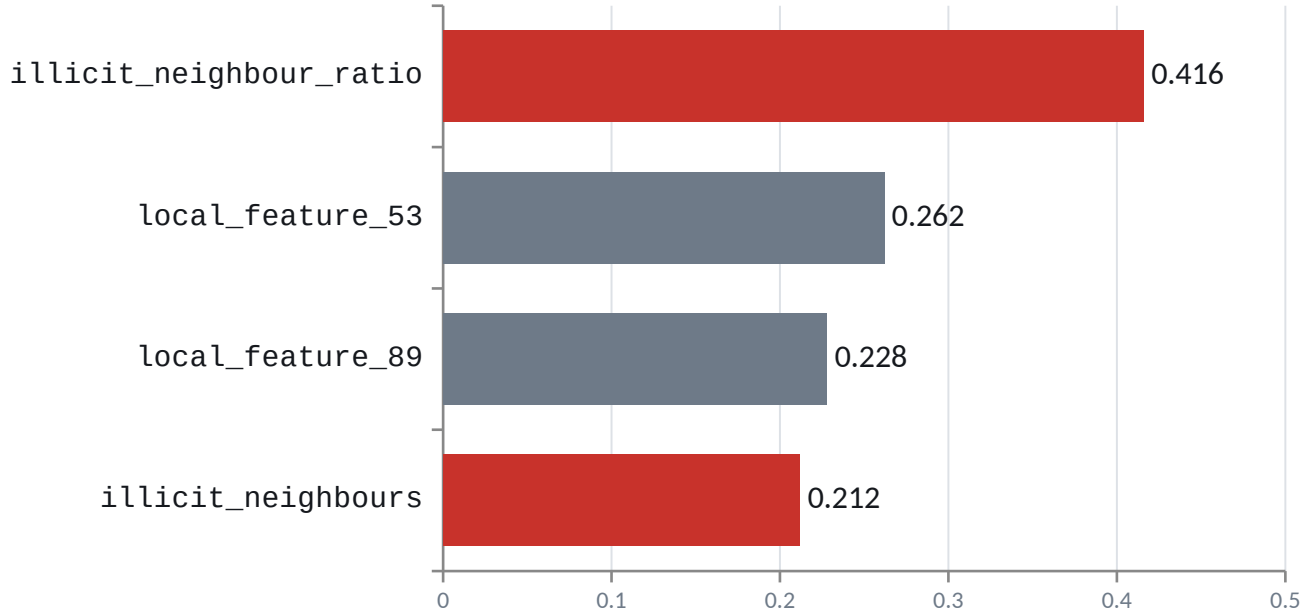


Distribution of effect size across all 165 features.

Feature	d	Effect
local_feature_53	-0.913	large
local_feature_89	-0.788	medium
local_feature_55	-0.786	medium
local_feature_90	-0.765	medium
aggregate_feature_49	0.659	medium
aggregate_feature_57	-0.642	medium

Signal is concentrated in about 5% of the columns. 102 of 165 separate the classes negligibly — they are retained in core, but a model given all 165 is mostly given noise.

The best predictor is one the database built



Absolute point-biserial correlation with the illicit outcome, on the 46,564 labelled transactions. Red bars are predictors derived from the edge table; grey bars shipped with the dataset.

Why this answers the question

illicit_neighbour_ratio did not come with the data. It exists because transactions and edges were modelled as separate relations that can be joined.

It is the strongest single predictor available — and the only strong one that can be explained to a compliance officer. "This transaction is adjacent to known illicit activity" is a defensible reason to review something. "local_feature_53 is low" is not.

Structuring the data did not merely make analysis faster. It produced the interpretable signal that the raw data could not supply.

The extract a regression would be fitted on

```
CREATE VIEW analytics.vw_analysis_matrix AS
SELECT t.tx_id, t.time_step,
       CASE WHEN t.time_step <= 34 THEN 'train'
            ELSE 'test' END   AS split,
       (t.class_id = 1)::INT  AS is_illicit,
       d.in_degree, d.out_degree,
       e.illicit_neighbours,
       e.illicit_neighbour_ratio
FROM   core.transaction t
JOIN   analytics.mv_tx_degree        d USING (tx_id)
JOIN   analytics.mv_illicit_exposure e USING (tx_id)
WHERE  t.class_id IN (1, 2);
```

Three things settled here rather than left to whoever runs the model: unlabelled rows are excluded rather than coerced to zero; the target is encoded once; the split is defined in the database, so every future analysis inherits the same one.

46,564

labelled rows, 11 columns — the analysis-ready table, produced by SQL alone.

Split	Periods	Rows	Illicit
train	1–34	29,894	11.58
test	35–49	16,670	6.50

Split by time, not at random

Edges never cross periods, and each period is its own connected component. A random split would let a model observe the future of the same component it is scoring. Periods 1–34 / 35–49 is the split used in the published literature on this dataset, which keeps results comparable.

The two sides are not the same regime — 11.58% illicit against 6.50%. That is reported, not resampled away: a monitoring system is judged on whether it holds up when the population shifts, which is precisely what periods 43–46 did.

INTERPRETATION

What the data supports — and what it does not

SUPPORTED	Counterparty adjacency carries real signal. 30.0% of illicit transactions have an illicit neighbour against 2.4% of licit ones, and the derived ratio correlates with the outcome at r = 0.416 — stronger than any of the 165 supplied features.
SUPPORTED	The illicit population is not stationary. A 40-fold collapse across periods 43–46 followed by recovery means any fixed threshold calibrated before period 42 would have behaved differently after it.
NOT SUPPORTED	Any statement about what drives illicit activity. The features are obfuscated: local_feature_53 separates the classes, but what it measures is unknown and unknowable from this data.
NOT SUPPORTED	Any attribution to a person, wallet or institution. Edges join transactions, not parties. Nothing here identifies who did anything.

Every claim above traces to a named object and a query that can be re-run — and every claim that cannot is listed as unsupported.

LIMITATIONS

Four constraints, scoped rather than concealed

Features are obfuscated

165 columns cannot be mapped to transaction value, fee or output count. Findings are stateable as "feature 53 separates the classes", never in business terms. Recorded in dim_feature.is_obfuscated.

Edges join transactions, not parties

No wallets, accounts or counterparties exist in the data, so no customer dimension can be built and no predicted-illicit transaction can be attributed to a responsible address.

77% unlabelled, 2.2% illicit

Supervised scope is confined to 46,564 rows. At that base rate accuracy is meaningless; any threshold rule needs precision and recall at a fixed review budget.

Labels are binary and historical

Scams, ransomware and terrorist financing collapse into one value, and the window closes before 2019 — so conclusions are methodological, not claims about current threat patterns.

None of these is fatal to the research question, which is about structure rather than prediction — but each one bounds what the finished database is allowed to claim.

REMAINING WORK

What Unit 10 delivers

01	**Fit and evaluate**	Fit a model on vw_analysis_matrix and report precision and recall at a fixed review budget — the metric a triage queue is actually judged on.
02	**Partition by period**	Range-partition core.transaction and tx_feature on time_step. Period-scoped queries are the dominant access pattern and currently scan more than they need.
03	**Refresh strategy**	Materialised views are rebuilt wholesale. Define incremental refresh and state its cost per period.
04	**Actor layer**	Extend the schema with the Elliptic++ address layer so exposure can be attributed to an entity rather than a transaction.
05	**Access control**	Role separation and row-level security, so the governance argument is enforced by the database rather than described in a document.

Every artefact supporting this milestone is itemised on the next slide.

What ships with this milestone

01	**elliptic_txs_*.csv**	Raw data — the three Elliptic files (classes, edgelist, features) in ./data/, 690 MB in total. A SHA-256 of each is recorded in governance.etl_load_audit before a single row is read.
02	**sql/01–04_*.sql**	SQL code — 01_schema.sql, 02_indexes.sql, 03_analytics.sql, 04_analysis.sql. 01 is emitted by etl/gen_schema.py, so the schema cannot drift from what was built.
03	**etl/etl_load.py**	ETL pipeline — extract, validate, transform, load and build, all in a single transaction. Idempotent: a fifth run produces the same database as the first.
04	**elliptic_aml.dump**	Database backup — pg_dump taken after a full validated run on PostgreSQL 16.2. 367 MB, of which core.tx_feature is 271 MB. Restorable without re-running the load.
05	**validation_run_log.txt**	Validation log — all nine rule results plus the injected-defect run; with docs/data_dictionary.md, read back out of the live catalog after loading.

Reproduce in three commands: `createdb elliptic_aml` · `psql -f sql/01_schema.sql` · `python3 etl/etl_load.py --data-dir ./data --dsn $DSN`

CONCLUSION

Answering the question set in Unit 1

01	**The question**	How can transaction-level Bitcoin data be structured in an RDBMS to support efficient identification and monitoring of patterns associated with illicit activity?
02	**What the structure made queryable**	A self-join through the edge table: 30.0% of illicit transactions sit next to a known illicit one against 2.4% of licit ones, and illicit_neighbour_ratio correlates with the outcome at r = 0.416.
03	**What it surfaced without a model**	A GROUP BY over one column exposed the 40-fold collapse across periods 43–46. The 46,564-row analysis matrix and its time-based split are defined in the database, not left to whoever fits the model.
	The bottom line	Structuring the data did not merely make analysis faster — it produced the interpretable signal the raw data could not supply. Every claim traces to a named object and a query that can be re-run.

Unit 10: fit and evaluate · partition by period · incremental refresh · actor layer · access control